

GreenByte — Syngenta Hackathon Demo Architecture

Document version: 1.2
Date: 2026-09-29
Team: GreenByte (HatchWorks AI Hackathon — AgTech Edition)
Status: Proposed baseline for UC1 (Plant Capacity) and UC4 (R&D Data Unification)

1. Purpose

This document describes the **integration architecture** for the Syngenta use-case demo:

- **Frontend** and **BFF** live in the GreenByte repository (`React` + `core-api` Serverless).
- **Data API** and **Agent API** may run on **separate platforms** (team specialization), exposed to the BFF via **HTTP/JSON**.
- **No live connections** to Syngenta production systems; all data comes from official hackathon extracts or synthetic files in the repo.

2. Acronyms and definitions

2.1 General and technical

Acronym	Definition
AgTech	Agricultural technology; digital tools and data applied to farming and the seed supply chain.
API	Application Programming Interface; HTTP endpoints that exchange JSON (or agreed formats) between services.
AWS	Amazon Web Services; cloud platform used by the GreenByte preset (e.g. Lambda, API Gateway, S3).
BFF	Backend for Frontend; a dedicated API (<code>core-api</code>) that orchestrates calls to Data and Agent services and exposes one contract to React.
CORS	Cross-Origin Resource Sharing; browser rules for which APIs the frontend may call; avoided by routing through the BFF.
CSV	Comma-Separated Values; UC4 hackathon datasets (<code>trial_synthetic.csv</code> , etc.).
ETL	Extract, Transform, Load; one-time or batch jobs that load Excel/CSV into the demo database.
GenAI	Generative AI; LLM-based generation and reasoning (central hackathon requirement).
GUID	Globally Unique Identifier; stable keys in UC4 data (<code>TRIAL_GUID</code> , <code>MATERIAL_GUID</code> , <code>LOCATION_GUID</code>).
HTTP/JSON	Hypertext Transfer Protocol with JavaScript Object Notation payloads between services.
LLM	Large Language Model; model used by the Agent API for NL answers and explanations.
MCP	Model Context Protocol; pattern of giving an agent structured tools (here implemented as Data API HTTP calls).
MSW	Mock Service Worker; frontend mocks that mirror BFF responses for offline or fallback demo.

NL	Natural language; breeder or scheduler questions phrased in plain text.
OpenAPI	Machine-readable API specification (Swagger); shared contract for BFF and integrations.
PG	PostgreSQL; recommended demo relational database (diagram shorthand).
RAG	Retrieval-Augmented Generation; LLM answers grounded in retrieved facts from the Data API (not free-form guessing).
RDS	Amazon Relational Database Service; managed PostgreSQL in AWS (optional; often skipped during hackathon week).
SQL	Structured Query Language; queries executed by the Data API against PostgreSQL.
SSM	AWS Systems Manager Parameter Store; store for non-secret and secret config (preferred over committing keys).
UC1	Use Case 1 — Plant Capacity Utilization (Pasco conditioning plant scheduling).
UC4	Use Case 4 — R&D Data Source Unification (trials, pedigree, field/lab observations).
UI	User interface; React demo pages under <code>/demo/plant</code> or <code>/demo/breeding</code> .

2.2 Syngenta and business context

Acronym	Definition
BU	Business Unit; Syngenta Vegetable Seeds BU for this hackathon.
CRM	Customer Relationship Management; commercial systems (out of scope for live connection).
ERP	Enterprise Resource Planning; e.g. SAP production orders in UC1 extracts (demo data only).
PO	Production Order; plant batch identifier (e.g. in Pasco schedules and logs).
QA	Quality Assurance; seed conditioning test outcomes (pass/fail logs in UC1).
R&D	Research and Development; breeding, trials, and advancement decisions (UC4).
R / A / G	Red, Amber, Green; triage recommendation levels for candidate lines (UC4 brief).
SAP	ERP system referenced in UC1 scheduling extracts (Excel SAP data sheet).
SME	Subject Matter Expert; Syngenta contacts listed per use case in the briefs.

2.3 Pasco plant data (UC1)

Acronym	Definition
LSV	Large Seed Vegetables; Pasco conditioning scope (e.g. sweet corn lines).
SSV	Small Seed Vegetables; Pasco conditioning scope (e.g. smaller-seed crops).
Conditioning	First processing stage after harvest: cleaning, sizing, and grading raw seed before treatment and packing.

2.4 UC4 data concepts

Term	Definition
Trial	A structured field or lab experiment (<code>trial_synthetic.csv</code>) with type such as yield, quality, or disease.
Material	A germplasm line or variety candidate (<code>germplasm_pedigree_synthetic.csv</code>).
Pedigree	Parental lineage and advancement history for a material.
Observation	Measured trait values from field (<code>observation_synthetic.csv</code>) or lab (<code>lab_observations_synthetic.csv</code>).
Operation	Field activity such as planting, harvest, or irrigation (<code>operations_synthetic.csv</code>).
Dossier	Aggregated UI view of one material's trial, obs, lab, and pedigree facts.

3. Hackathon constraints (from briefs)

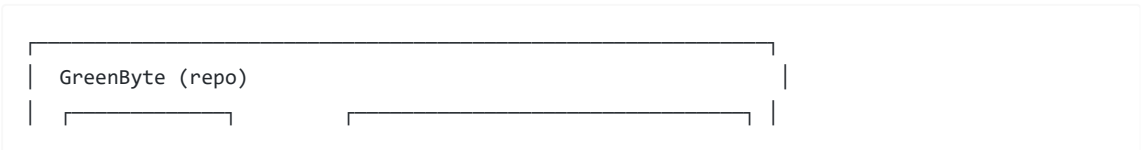
Constraint	Implication
GenAI is central	Agent API owns LLM orchestration; not decorative copy on static rules
Human in the loop	UI validates or overrides; audit trail where applicable
No production SAP/CRM/R&D systems	ETL from local Excel/CSV into demo database
Recommendations with explanations	BFF returns structured reasons, not opaque scores

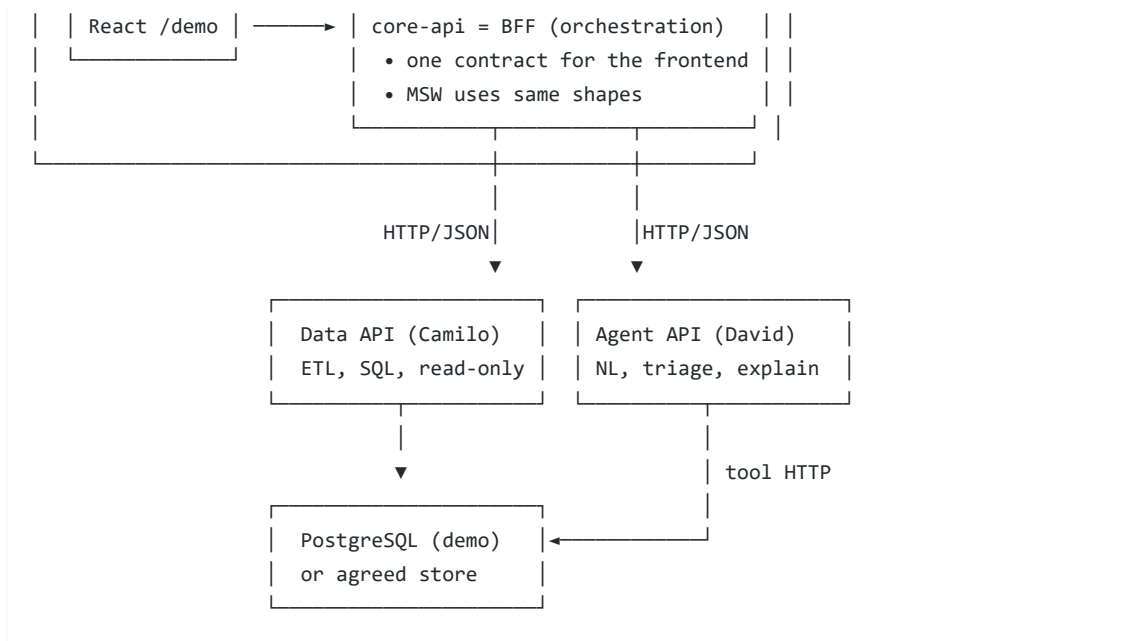
4. Team responsibilities (integration view)

Role	Owner	Responsibility
Product UI + BFF	Mauricio / GreenByte frontend	Single API contract to React; CORS; MSW mocks; optional override persistence
Data API	Camilo	ETL, schema, read-only query/tools endpoints, data quality
Agent API	David	Chat, triage, explain, tool-use calling Data API URLs

The **React app must not call Data or Agent APIs directly** in the demo environment (only the BFF), to simplify CORS, secrets, and fallback to MSW.

5. Common container diagram





5.1 GreenByte stack (existing preset)

Layer	Technology
Frontend	React, TypeScript, Vite, Tailwind, MSW, axios
BFF	AWS Lambda, API Gateway, Node.js 20 (core-api)
Shared	layer-transversal (consistent JSON responses)
Secrets	SSM / Secrets Manager (no keys in repo)
Region	us-east-1

5.2 Recommended demo data store

- **PostgreSQL** (local Docker under `local-dev/` or team-hosted) with seed scripts under `backend/database/`.
- RDS is optional for hackathon week; cost and setup time may be prohibitive.
- Agent **must not** read raw CSV/Excel at runtime in production demo path; only **Data API** reads the database.

6. Integration rules (day 1)

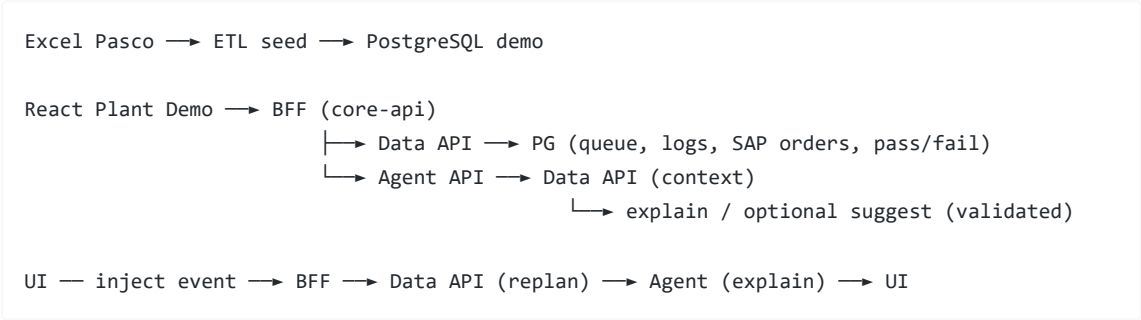
1. **Contract first:** OpenAPI or shared JSON examples for BFF ↔ frontend and BFF ↔ Data/Agent.
2. **Parallel mocks:** MSW handlers mirror BFF responses for offline demo.
3. **Read-only tools:** Data API exposes parameterized queries (limits, allowed filters).
4. **Agent grounding:** LLM context built from Data API JSON only; citations reference stable IDs (`PO Number` , `MATERIAL_GUID` , etc.).
5. **Timeouts and fallback:** BFF degrades gracefully (cached queue or scripted answer) if Agent is unavailable.

7. Use Case 1 — Plant Capacity Utilization (Pasco)

7.1 Business flow

- 1. Scheduler views **line queue** (batches, priority, due dates).
- 2. **Event** injected: rush batch or failed QA.
- 3. **Replan** updates order (heuristics in Data API or BFF).
- 4. **Agent** explains what changed in plain language.
- 5. Human **accepts** the recommendation (no auto-write to ERP).

7.2 Architecture flow



7.3 Suggested endpoints (illustrative)

BFF (frontend contract)

Method	Path	Description
GET	/demo/plant/lines/{lineId}/queue	Current ranked queue
POST	/demo/plant/events	Rush batch or QA failure
POST	/demo/plant/schedule/accept	Human accepts plan
GET	/demo/plant/batches/{po}/summary	Batch context for panel

Data API (Camilo)

Method	Path	Description
GET	/lines/{id}/queue	Queue from seeded Pasco data
POST	/schedule/replan	Heuristic reorder after event
GET	/batches/{po}	Species, kg, history, QA flags

Agent API (David)

Method	Path	Description
POST	/explain-replan	Natural-language diff given structured payload
POST	/suggest-rank	Optional; BFF validates against rules

7.4 Data sources (repo)

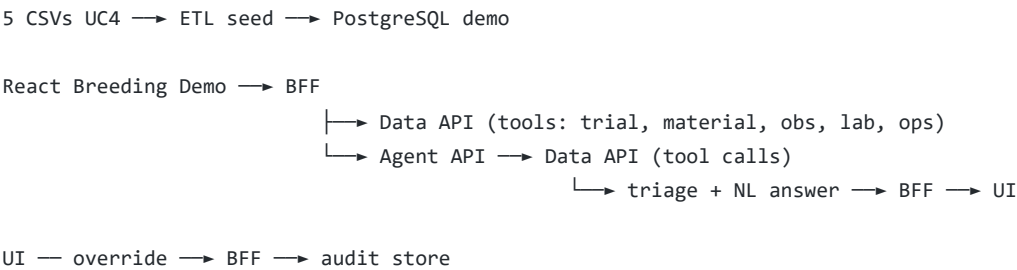
Key sheets: line schedules, SAP orders, conditioning logs, pass/fail logs.

8. Use Case 4 — R&D Data Source Unification

8.1 Business flow

1. Breeder asks a **natural-language question** across trials, field obs, lab, pedigree, operations.
2. **Agent** calls **read-only tools** on Data API (MCP-style over HTTP).
3. System returns **red / amber / green** triage with **stated reasons**.
4. Breeder **overrides**; override is **logged** (BFF or Data API).

8.2 Architecture flow



8.3 Suggested endpoints (illustrative)

BFF

Method	Path	Description
POST	/demo/breeding/ask	NL question → answer + citations + RAG
GET	/demo/breeding/materials/{guid}/dossier	Aggregated view for UI
POST	/demo/breeding/recommendations/{id}/override	Human override + audit

Data API (tools, read-only)

Method	Path	Description
GET	/trials/{trialGuid}	Trial metadata
GET	/materials/{materialGuid}/pedigree	Pedigree and advancement
GET	/materials/{materialGuid}/observations	Field + lab obs
GET	/trials/{trialGuid}/operations	Field operations

Agent API

Method	Path	Description
--------	------	-------------

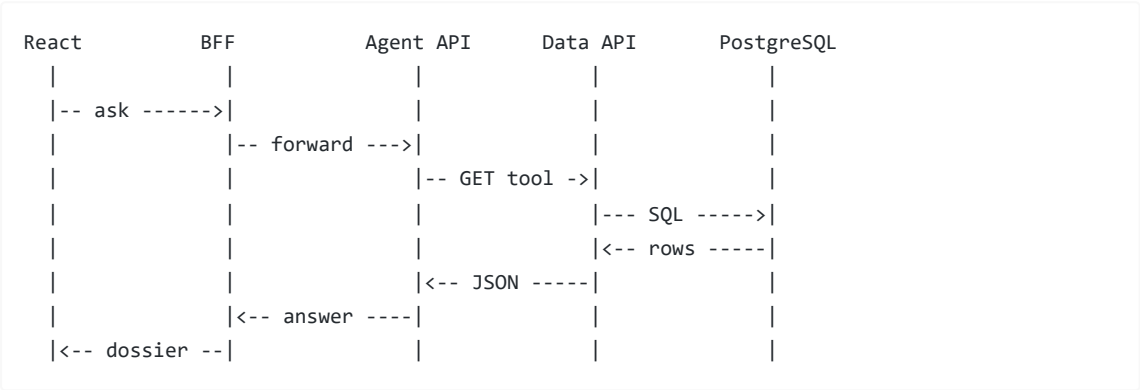
POST	/chat	Orchestrates tool calls to Data API
POST	/trriage	R/A/G with reason lines

8.4 Data sources (repo)

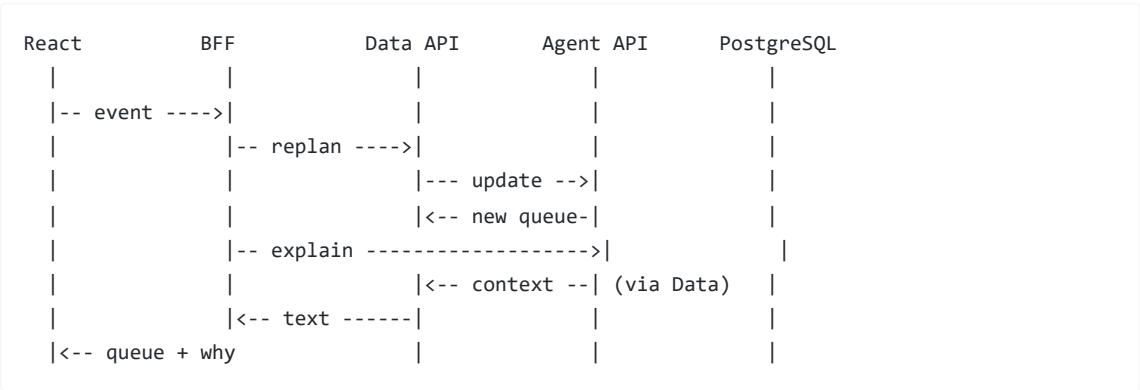
Hackathon 2026 - Use Cases/.../UC4 - R&D Data Source Unification/*.csv

Relational keys: TRIAL_GUID , MATERIAL_GUID , LOCATION_GUID .

9. Sequence — UC4 (ask + tools)



10. Sequence — UC1 (event + replan + explain)



11. Monolith vs distributed (documentation levels)

Audience	Diagram
Judges / business	Single “copilot over demo data” story is enough
Team execution	Three services + BFF (this document)
Handoff to Syngenta	OpenAPI for BFF + list of Data/Agent base URLs per environment

Legacy single-box diagrams (everything inside `core-api`) remain valid as **logical** view; **physical** deployment follows Section 5.

12. Generic template (either use case)

```
Sources (Excel or CSV) --> ETL --> PostgreSQL demo
React GreenByte --> BFF core-api --> Data API --> PG
                  \--> Agent API --> Data API
                  --> UI
```

Replace **Sources** and **React** route name (`/demo/plant` vs `/demo/breeding`) per selected use case.

13. Next steps in the repo

1. Choose UC1 or UC4 and register a feature story under `cursor/analysis/features/` .
 2. Publish BFF OpenAPI under `backend/docs/api/` .
 3. Add MSW handlers matching BFF contracts in `frontend/src/mocks/` .
 4. Implement seed scripts from hackathon data paths.
 5. Wire environment variables for Data and Agent base URLs in BFF config only.
-

14. UI mockups (wow moment)

Use case	Mockup
UC1 — Plant Capacity	mockups/uc1-plant-capacity-wow.png
UC4 — R&D Unification	mockups/uc4-rd-unification-wow.png

See [mockups/README.md](#) for scene description and target routes.

References

- `cursor/company/HACKATHON.md` — event rules and evaluation criteria
- `docs/architecture.md` — GreenByte preset overview
- `Hackathon 2026 - Use Cases/.../2026_Use_Case_Briefs.pdf` — Syngenta success criteria
- `project.config.json` — stack and region metadata